Estructuras de Datos

2026
Segundo Cuatrimestre

REPASO

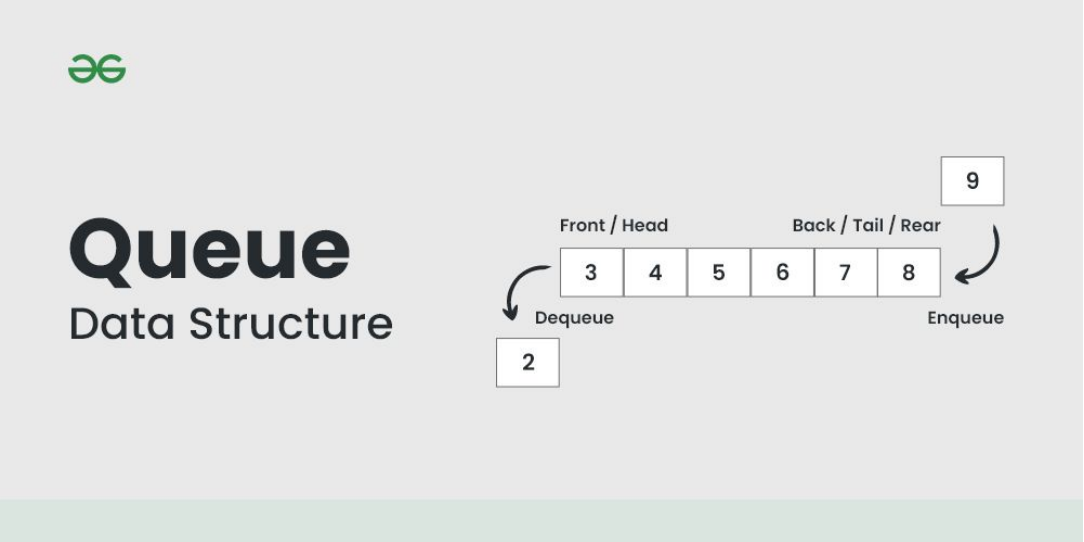
TDA Cola / ADT Queue



La primera persona en la cola, será la primera en ser atendida y saldrá de la cola.

La última persona en agregarse a la cola irá al final de la misma y será la última persona en salir de la cola.

TDA Cola / ADT Queue



La primera persona en la cola, será la primera en ser atendida y saldrá de la cola.

La última persona en agregarse a la cola irá al final de la misma y será la última persona en salir de la cola.



CLASIFICACION TRIAGE

NIVEL DE URGENCIA	TIPO DE URGENCIA	COLOR	TIEMPO DE ESPERA
1	RESUCITACION	ROJO	ATENCION DE FORMA INMEDIATA
2	EMERGENCIA	NARANJA	10 - 15 MINUTOS
3	URGENCIA	AMARILLO	60 MINUTOS
4	URGENCIA MENOR	VERDE	2 HORAS
5	SIN URGENCIA	AZUL	4 HORAS

TDA Cola con Prioridad

TDA Cola con Prioridad

Una cola con prioridad almacena una colección de elementos que soporta:

- Inserción de elementos arbitraria
- Eliminación de elementos en orden de prioridad (el elemento con 1era prioridad puede ser eliminado en cualquier momento)

Nota: Una cola con prioridad almacena sus elementos de acuerdo a su prioridad relativa y no expone una noción de “posición” a sus clientes.

TDA Cola con Prioridad

Prioridad: Atributo de un individuo que sirve para pesar al individuo en un conjunto de individuos.

Ejemplo: Promedio de un alumno para otorgar becas

Ejemplo: Gravedad de la situación de un paciente en una sala de espera para decidir a quién atender primero

TDA Cola con Prioridad

Una cola con prioridad es una colección de elementos, llamados valores, los cuales tienen asociada una prioridad que es provista en el momento que el elemento es insertado. Un par prioridad-valor insertado en un cola con prioridad se llama una entrada.

Operaciones fundamentales de una cola con prioridad P:

- insert(k, x): Inserta un valor x con prioridad k en P
- removeMin(): Retorna y remueve de P una entrada con la prioridad más pequeña
- min(): Retorna de P una entrada con la prioridad más pequeña sin removerla

TDA Cola con Prioridad

```
/** K representa el tipo de la prioridad del objecto de tipo V almacenado en la cola con prioridad*/
public interface PriorityQueue<K,V> {

    /** Retorna el número de ítems en la cola con prioridad. */
    public int size();

    /** Retorna si la cola con prioridad está vacía. */
    public boolean isEmpty();

    /** Retorna pero no elimina una entrada con minima prioridad. */
    public Entry<K,V> min() throws EmptyPriorityQueueException;

    /**Inserta un par clave-valor y retorna la entrada creada.*/
    public Entry<K,V> insert(K key, V value) throws InvalidKeyException;

    /** Remueve y retorna una entrada con minima prioridad. */
    public Entry<K,V> removeMin() throws EmptyPriorityQueueException; }
```

TDA Cola con Prioridad - Entradas

```
public interface Entry<K,V> {
        public K getKey(); // Retorna la prioridad de la entrada
        public V getValue(); // Retorna el valor de la entrada
}

public class Entrada<K,V> implements Entry<K,V> {
        private K clave;
        private V valor;
        public Entrada(K key, V value) { clave = key; valor = value; }
        public K getKey() { return clave; }
        public V getValue() { return value; }
        public void setKey( K key ) { clave = key; }
        public void setValue(V value) { value = value; }
        public String toString( ) {
                return “(“ + getKey() + “,” + getValue() + “)” ;
        }
}
```

Comparación de Elementos

Si a es una variable de un tipo que implementa la interfaz java.lang.Comparable, compareTo abstrae la comparación con otro b:

a.compareTo(b) =

- 0, si a == b
- entero negativo, si a < b
- entero positivo, si a > b

Comparación de Elementos

Un objeto c también puede ser un *comparador* que abstrae la noción de comparación entre dos objetos a y b de tipo genérico E.

Si c es una variable de tipo java.util.Comparator<E>, entonces c.compare(a, b) =

- 0, si a == b
- entero negativo, si a < b
- entero positivo, si a > b

Comparación de Elementos

Un objeto c también puede ser un *comparador* que abstrae la noción de comparación entre dos objetos a y b de tipo genérico E.

Si c es una variable de tipo java.util.Comparator<E>, entonces c.compare(a, b) =

El comparador por defecto delega en el comportamiento del tipo E implementado por la función compareTo del tipo E:

```
public class DefaultComparator<E> implements Comparator<E> {

    public int compare( E a, E b ) throws ClassCastException {

        return ((Comparable<E>)a).compareTo(b); } }
```

Comparación de Elementos

```
public class Persona implements
Comparable<Persona> {
        private String nombre;
        private float altura;

        public Persona(String nombre, float altura) {
                this.nombre = nombre; this.altura = altura; }

        public int compareTo(Persona p) {
                if (altura == p.getAltura()) return 0;
                else if (altura < p.getAltura()) return -1;
                else return 1; }

        public float getAltura() { return altura; } }
```

```
Persona p1 = new Persona("Juan", 1.80f);

Persona p2 = new Persona("Tito", 1.60f);

int r1 = p1.compareTo(p2);

Comparator<Persona> comp =

        new DefaultComparator<Persona>();

int r2 = comp.compare(p1, p2);
```

r1 y r2 = 1 porque p1 > p2 porque

altura(p1) > altura(p2)

Comparación de Elementos

Si ahora quiero comparar a las personas por edad, en el primer caso tengo que modificar la implementación del método compareTo de la clase persona, en cambio en el segundo caso implemento otro comparador.

```
public class miComparador<E> implements Comparator<E extends Persona> {

    public int compare( E a, E b ){

        return a.getEdad()-b.getEdad(); } }
```

Comparación de Elementos y CCP

Entonces, tenemos dos alternativas en cuanto a comparación de elementos y CCP:

- Usamos un elemento **X extends Comparable<E>** y delegamos la comparación en la comparación de cada elemento
- Parametrizamos la comparación teniendo un atributo de instancia **Comparator<E> comp**

Comparación de Elementos y CCP

Supongamos que en mi CCP quiero insertar personas:

-Si uso el comparador propio de Persona, la CCP se va a organizar de acuerdo a las alturas de las personas porque así fue definido en esa clase.

-Si quiero organizar las personas según su edad, ese enfoque no me sirve.

-Tener parametrizado el comparador, me da la libertad de cambiar la forma de comparar los elementos cuando lo deseo.

TDA Cola con Prioridad - Implementaciones

size();
isEmpty();
min()
insert()
removeMin()			

TDA Cola con Prioridad - Implementaciones

	Lista		

size();
isEmpty();
min()
insert()
removeMin()
	O(1)
	O(1)
	O(n)
	O(1)
	O(n)		

TDA Cola con Prioridad - Implementaciones

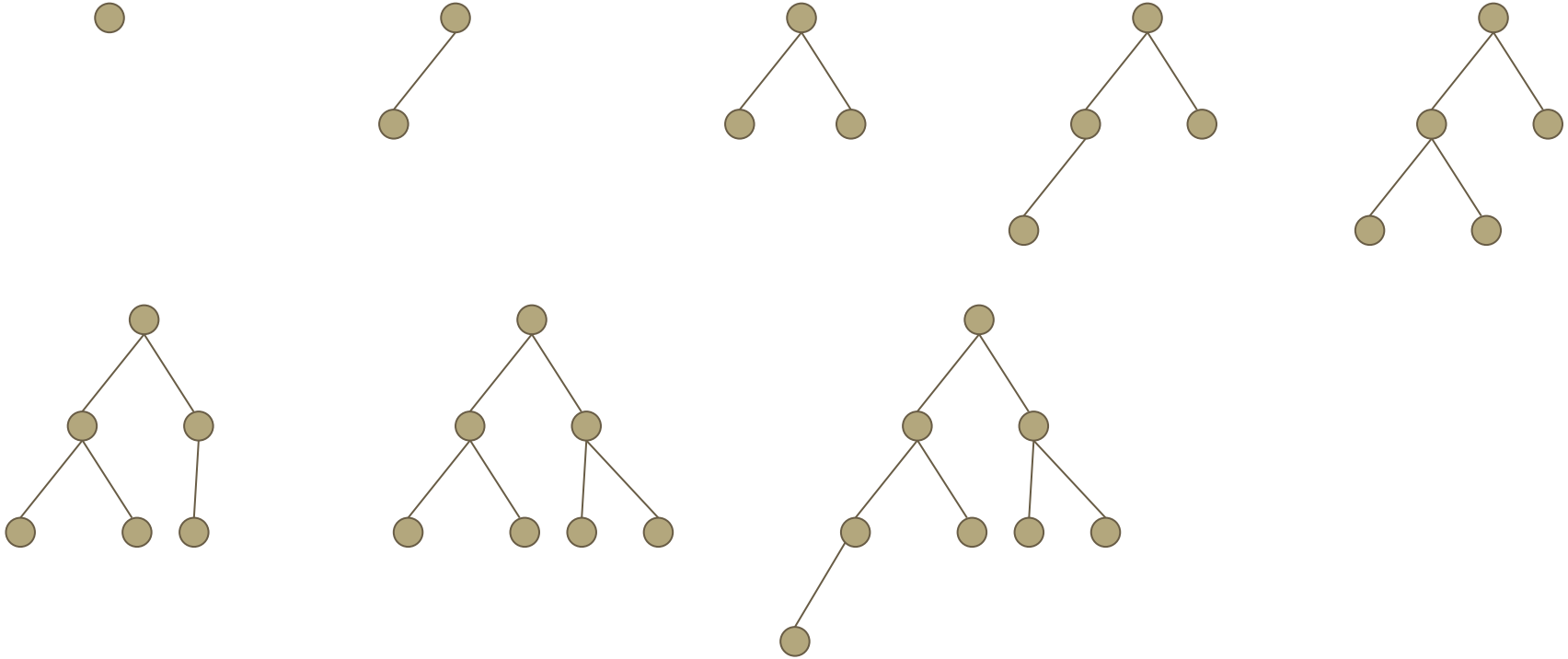
	Lista no ordenada	Lista ordenada	Heap
size();	O(1)	O(1)	
isEmpty();	O(1)	O(1)	
min()	O(n)	O(1)	
insert()	O(1)	O(n)	
removeMin()	O(n)	O(1)	

TDA Cola con Prioridad - HEAP

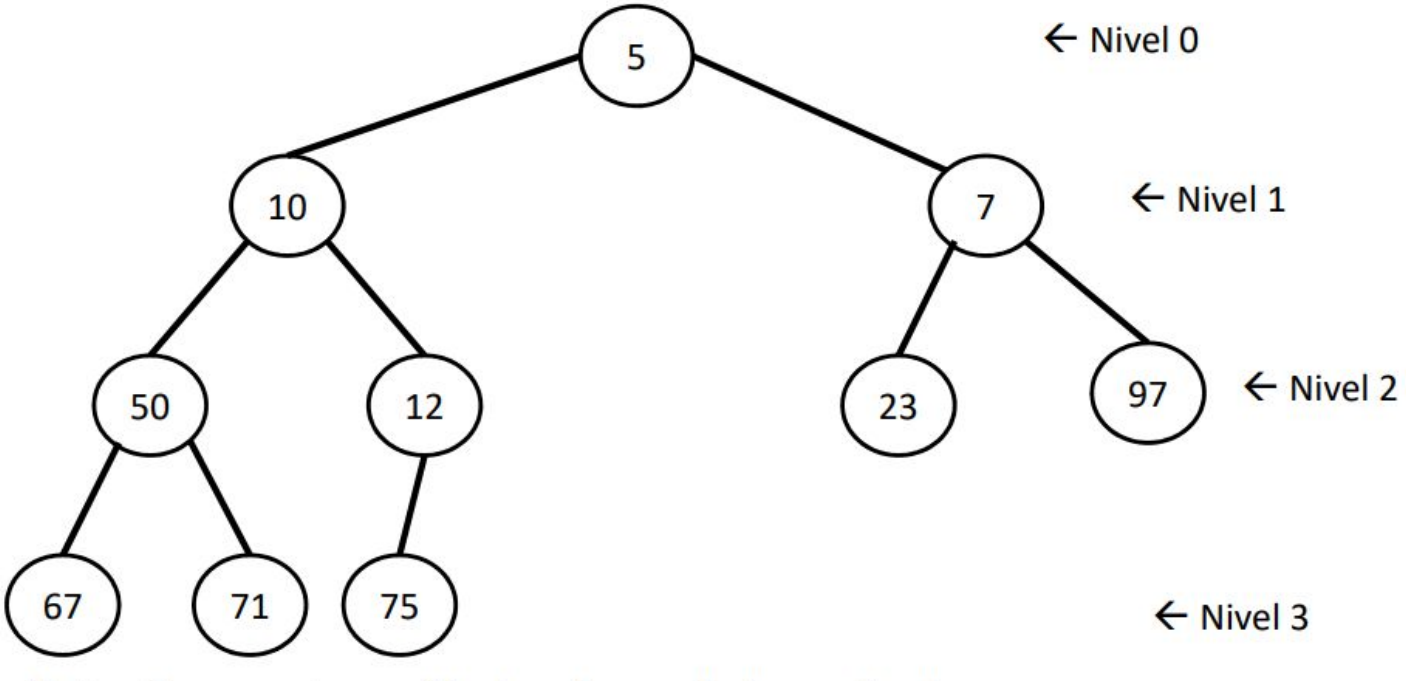
Un (mín)heap es un árbol binario que almacena una colección de entradas en sus nodos y satisface dos propiedades adicionales:

- Propiedad de orden del heap (árbol parcialmente ordenado): En un heap T, para cada nodo v distinto de la raíz, la clave almacenada en v es mayor o igual que la clave almacenada en el padre de v.
- Propiedad de árbol binario completo: Un heap de altura h es un árbol binario completo si todos los niveles 0,1,...,h−1 contienen el máximo número posible de nodos, y los nodos del nivel h ocupan las posiciones de más a la izquierda. Por lo tanto, si existe un nodo con un único hijo en el nivel h-1, éste debe ser su hijo izquierdo, y dicho nodo es el nodo interno más a la derecha del árbol.

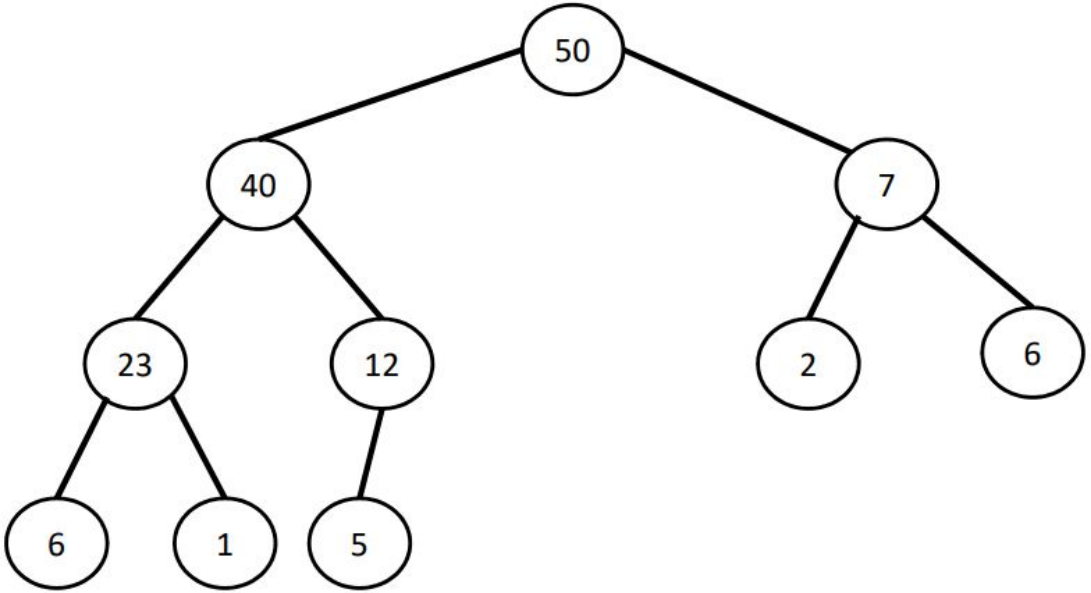
TDA Cola con Prioridad - HEAP



TDA Cola con Prioridad - (min)HEAP



TDA Cola con Prioridad - (max)HEAP



HEAP implementación AB con arreglos

La componente 0 no se usa

La raíz se almacena en la componente 1

El hijo izquierdo de la componente i se almacena en la componente 2*i

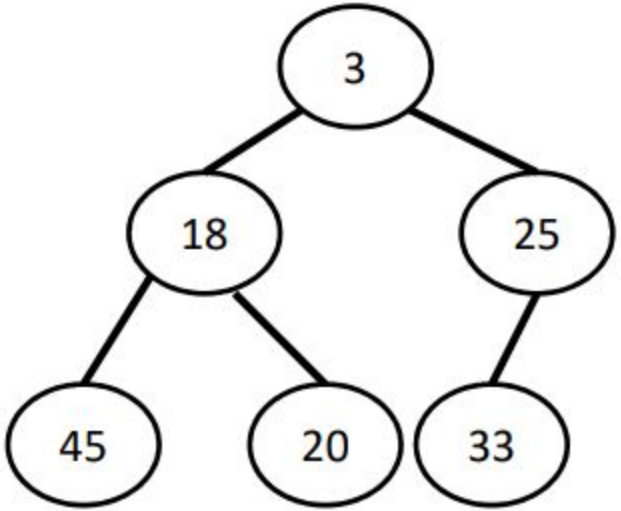
El hijo derecho de la componente i se almacena en la componente 2*i + 1

El padre de i está en la componente i / 2

Hijo_izquierdo(i) = 2i

Hijo_derecho(i) = 2i+1

Padre(i) = i div 2



0	1	2	3	4	5	6	7	8	9	10	11	12	13
	3	18	25	45	20	33							

TDA Cola con Prioridad - Implementación

Heap con Arreglo

```
public class Heap<K,V> implements PriorityQueue<K,V> {
    protected Entrada [] elems;
    protected Comparator<K> comp;
    protected int size;
```

TDA Cola con Prioridad - Implementación

```
public Heap(int maxElems, Comparator<K> comp ) {
        elems = (Entrada<K,V> []) new Entrada[maxElems];
        this.comp = comp;
        size = 0;
        }
```

TDA Cola con Prioridad - Implementación

Insertar

- El nuevo elemento se pone al final del árbol
- Mientras el hijo sea menor que el padre, se intercambian
    - Se corta
        - Cuando el padre es más chico que el hijo
        - Cuando se llega a la raíz

TDA Cola con Prioridad - Implementación

Eliminar

- Se saca el elemento en la raíz del árbol
- Se ubica el último elemento del árbol en el lugar de la raíz
- Se empieza a empujar hacia abajo por el hijo más chico
    - Se corta
        - Cuando se llega al final del árbol
        - Cuando los hijos son más grandes que el padre

TDA Cola con Prioridad - Implementaciones

	Lista no ordenada	Lista ordenada	Heap
size();	O(1)	O(1)	O(1)
isEmpty();	O(1)	O(1)	O(1)
min()	O(n)	O(1)	O(1)
insert()	O(1)	O(n)	O(log n)
removeMin()	O(n)	O(1)	O(log n)

HEAP SORT con CCP

```
function heapsort(array A[0..n]):
  CCP // creo una CCP implementada con heap
  integer i; // declaro variable i
  for i = 0..n:
    CCP.insert(A[i])
  for i = 0..n:
    A[i] = CCP.removeMin()
```

O(n log n) ¿Por qué?

TDA Cola con Prioridad

Capítulo 8 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.